COA_UNIT - III

ADITYA UNIVERSITY

Central Processing Unit:

- General Register Organization
- Stack Organization
- Instruction Formats
- Addressing Modes
- Data Transfer and Manipulation
- Program Control
- Reduced Instruction Set Computer

Microprogrammed Control:

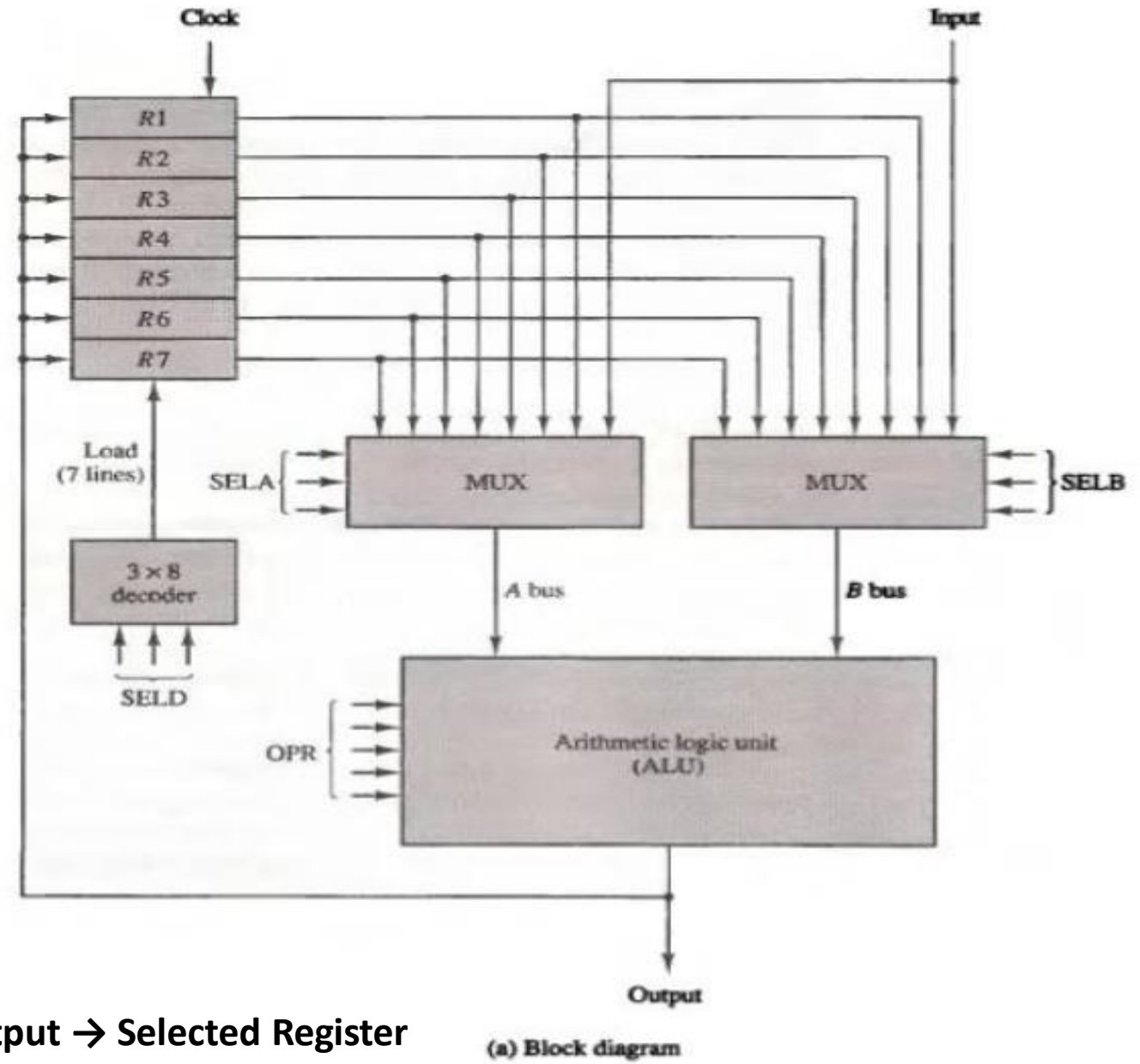
- Control Memory
- Address Sequencing
- Design of Control Unit
- Microprogram Example

General Register Organization

- The system consists of **7 registers (R1 to R7)** connected to a **common data path**.
- Two **multiplexers (MUX)** are used to select data from registers using SELA and SELB.

  SELA selects input for the **MUX A**

  SELB selects input for the **MUX B**
- Selected register values are sent to ALU through **A bus** and **B bus.**
- The **ALU** performs arithmetic/logic operations based on **OPR (operation select lines)** and the result is produced at the **output**.
- The output value can also load into Registers by using a 3x8 decoder controlled by SELD.
- All register operations are synchronized using the **clock signal**.



- **FLOW: Registers → MUX → A & B buses → ALU → Output → Selected Register**

General Register Organization – Control Word and Encoding

Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

For example: subtraction microoperation given by the statement $R_1 \leftarrow R_2 - R_3$

Field	SEL A	SEL B	SEL D	OPR
Symbol	R2	R3	R1	SUB
Control word	010 011 001 00101			

Encoding of ALU operations

OPR Select	Operation	Symbol
00000	Transfer *A*	TSFA
00001	Increment *A*	INCA
00010	Add *A* + *B*	ADD
00101	Subtract *A* − *B*	SUB
00110	Decrement *A*	DECA
01000	AND *A* and *B*	AND
01010	OR *A* and *B*	OR
01100	XOR *A* and *B*	XOR
01110	Complement *A*	COMA
10000	Shift right *A*	SHRA
11000	Shift left *A*	SHLA

Stack Organization

- A **stack** is a special type of storage organization and it follows: **LIFO (Last In First Out).**
- **LIFO (Last In First Out)** means, the data entered last will come out first.
- Two basic operations are used to organize the stack:

  PUSH → insert an item into the stack:

  POP → remove an item from the stack
- **A stack pointer (SP)** is used to indicate the topmost element of the stack.
- **Types of Stack:** There are two types of stack organization:

  1. **Register Stack**
  2. **Memory Stack**

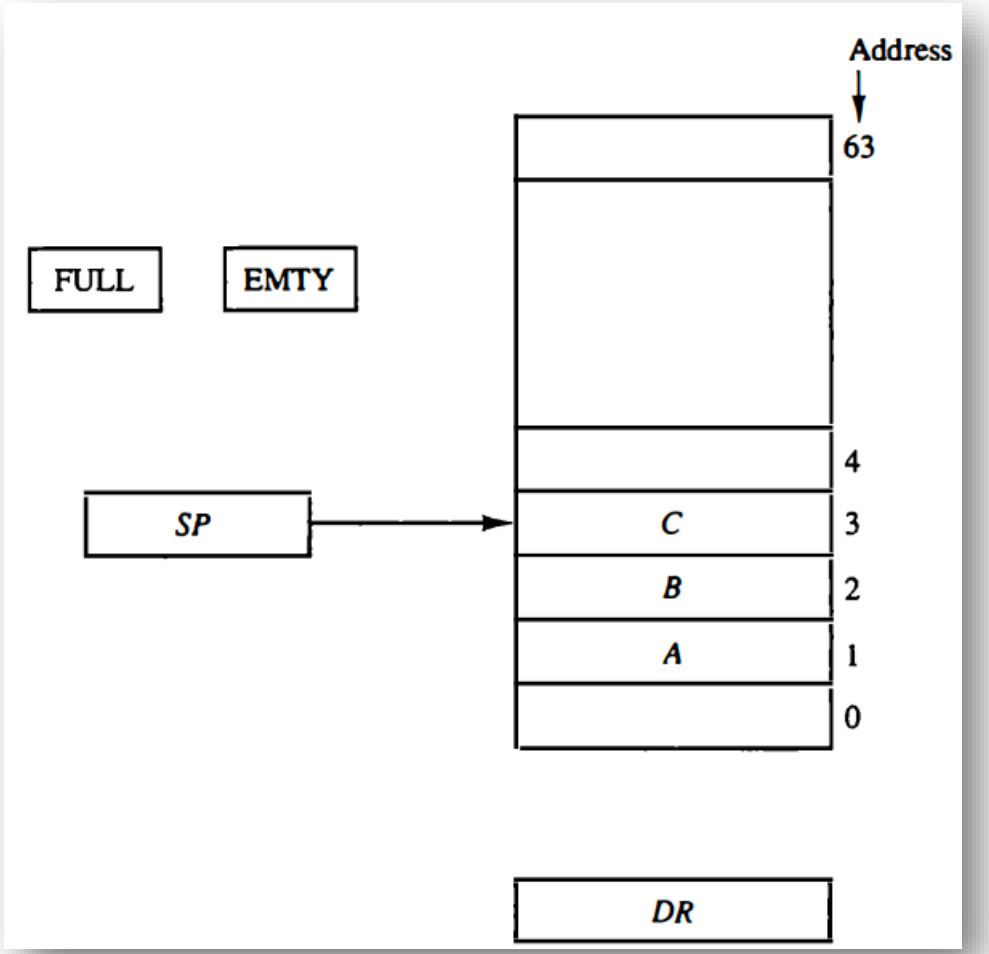
Register Stack

Register Stack:

- A **register stack** is a stack implemented using a set of CPU registers.
- The stack pointer (SP) indicates which register is at the top.
- The figure shows the organization of a 64-word register stack.
- **PUSH Operation:** (Initially FULL ← 0 & EMTY ← 1)

  The push operation is implemented with the following sequence of microoperations:

SP ← SP + 1	Increment stack pointer
M [SP] ← DR	Write item on top of the stack
If (SP = 0) then (FULL ← 1)	Check if the stack is full
EMTY ← 0	Mark the stack not empty



Register Stack

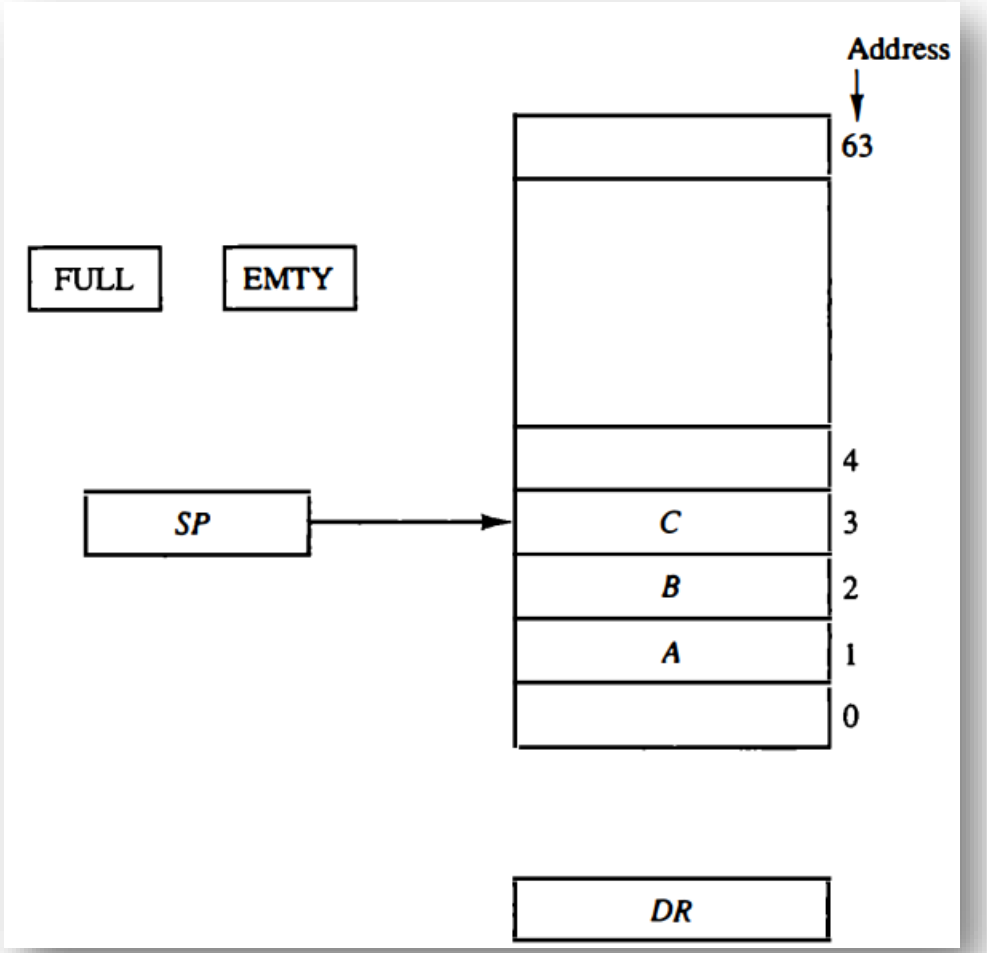
- **POP Operation:**
  - An item is removed from the stack if the stack is not empty (if EMTY = 0).
  - The push operation is implemented with the following sequence of microoperations:

$DR \leftarrow M[SP]$ Read item from the top of the stack

$SP \leftarrow SP - 1$ Decrement stack pointer

$IF(SP = 0)\ then\ (EMTY \leftarrow 1)$ Check if stack is empty

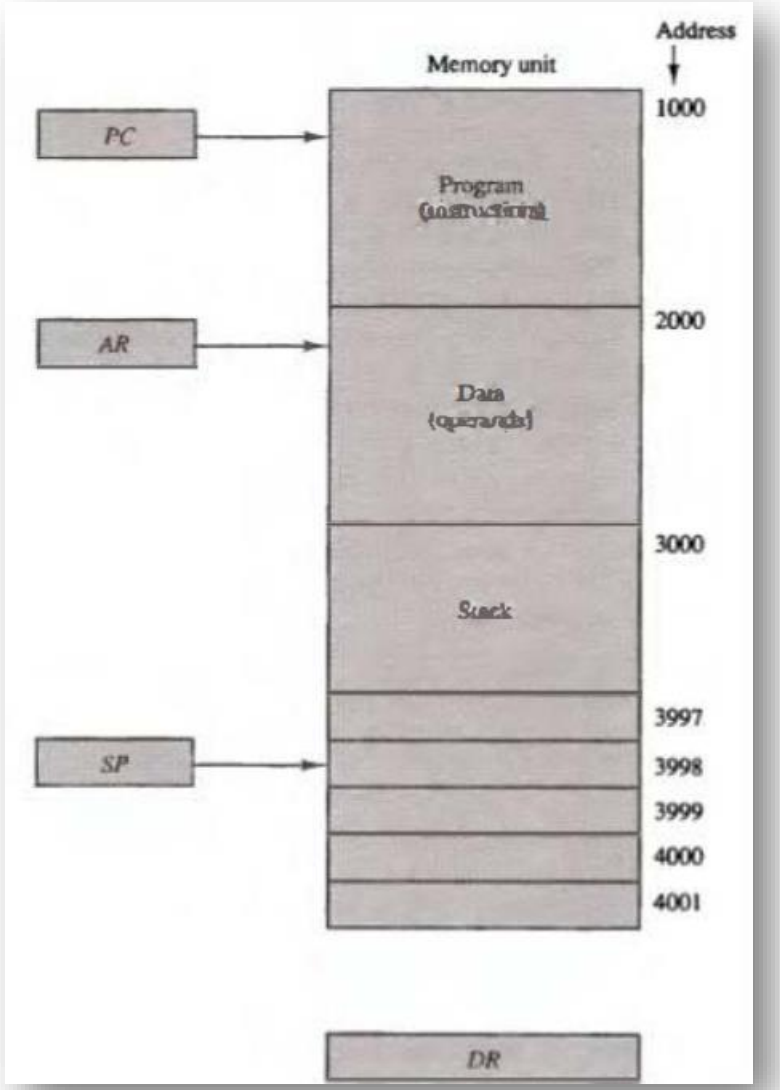
$FULL \leftarrow 0$ Mark the stack not full



Memory Stack

Memory Stack:

- The computer memory partitioned into three segments: program, data, and stack.
- The program counter PC points at the address of the next instruction in program.
- The address register AR points at an array of data.
- The stack pointer SP points at the top of the stack.
- As shown in the Figure, the initial value of SP is 4001, and the stack grows with decreasing addresses.
- Thus, the first item stored in the stack is at address 4000, the second item is stored at address 3999, and the last address that can be used for the stack is 3000.
- A new item is inserted with the push operation as follows:
  - $SP \leftarrow SP - 1$
  - $M[SP] \leftarrow DR$
- A new item is deleted with a pop operation as follows:
  - $DR \leftarrow M[SP]$
  - $SP \leftarrow SP + 1$



Reverse Polish Notation (RPN)

- **Reverse Polish Notation (RPN)** is a way of writing arithmetic expressions in which the operator comes after the operands. It is also called Postfix Notation.
- **Basic Idea**

  Infix (normal form): A + B

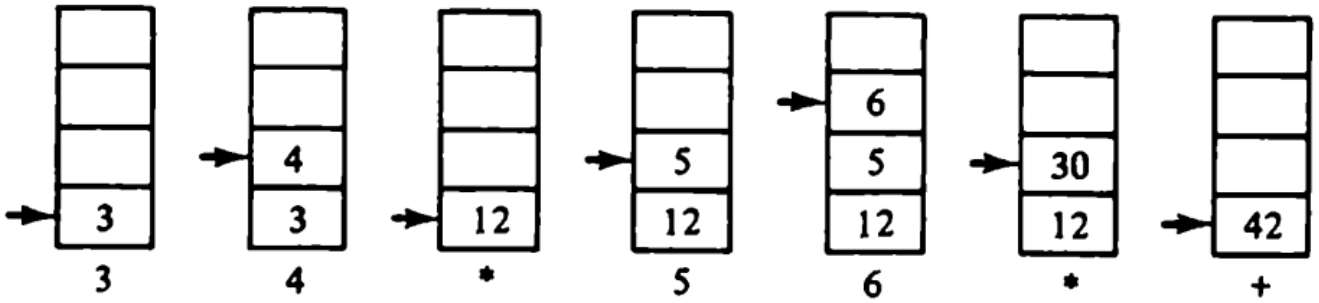
  Postfix (RPN): A B +

  No need for brackets or precedence rules, because the order of operations is clear.
- **Examples :**

Infix (Normal form):	Postfix (RPN)
A + B	A B +
(A + B) × C	A B + C ×
A × B + C	A B × C +
A + B × C	A B C × +

Stack operation in Reverse Polish Notation (RPN)

- **Example:** Evaluate **(3 × 4 + 5 × 6)** using stack operations in **Reverse Polish Notation.**
- To evaluate **(3 × 4 + 5 × 6)** using stack operations in **Reverse Polish Notation**, first write the expression in postfix form: **3 4 × 5 6 × +**



- **Simple steps form:**
  - Push **3**
  - Push **4**
  - Multiply top two values: **3 × 4 = 12**, push **12**
  - Push **5**
  - Push **6**
  - Multiply top two values: **5 × 6 = 30**, push **30**
  - Add top two values: **12 + 30 = 42**, push **42**
  - So, the final answer is **42**.

ADITYA UNIVERSITY

Instruction Formats

- An instruction format defines how an instruction is represented in binary.
- It consists of **Opcode** and **Address field**.

  Opcode: It specifies the operation to be performed

  Address fields: It specifies the operand location or Adress
- The instruction format may consist of multiple address fields.
- There are five types of Instruction formats:
  1. Three address Instruction format
  2. Two address Instruction format
  3. One address Instruction format
  4. Zero address Instruction format
  5. RISC Instruction format

Instruction Formats

Three address Instruction format:

- An instruction format that uses three addresses to specify two operands and one result location.
- Format: OPCODE A B C

OPCODE	Adrees of Operand A	Adrees of Operand B	Adrees of Operand C

- Meaning: A = B OPCODE C
- Example: ADD R1, R2, R3; That is R1 = R2 + R3.

Two address Instruction format:

- An instruction format that uses two addresses where one operand also stores the result.
- Format: OPCODE A B

OPCODE	Adrees of Operand A	Adrees of Operand B

- Meaning: A = A OPCODE B
- Example: ADD R1, R2; That is R1 = R1 + R2

Instruction Formats

One address Instruction format:

- An instruction format that uses a single address along with an implicit accumulator register.
- Format: **OPCODE A**

OPCODE	Adrees of Operand A

- Meaning: **AC = AC OPCODE A**
- Example: **ADD A; That is AC = AC + A**

Zero address Instruction format:

- An instruction format that does not use any address field and operates on a stack.
- Format: OPCODE

OPCODE

- Meaning: Operands are taken from stack
- Example: A B +; That is A+B

Instruction formats

ADITYA UNIVERSITY

RISC Instruction Format:

- RISC (Reduced Instruction Set Computer) architecture uses **simple, fixed-length instructions** to achieve high-speed execution.

Types of RISC Instructions:

1. **LOAD Instructions**
   - Transfer data from memory to registers
   - Example: R1 ← M[A]
2. **STORE Instructions**
   - Transfer data from registers to memory
   - Example: M[X] ← R1
3. **Computational Instructions**
   - Perform operations using registers
   - Typically follow three-address format
   - Example: R1 ← R2 + R3

Instruction formats-Example Program for RISC:

To evaluate: X = (A + B) * (C + D)

Program Steps:

```
LOAD   R1, A      → R1 ← M[A]
LOAD   R2, B      → R2 ← M[B]
LOAD   R3, C      → R3 ← M[C]
LOAD   R4, D      → R4 ← M[D]

ADD    R1, R1, R2 → R1 ← R1 + R2
ADD    R3, R3, R4 → R3 ← R3 + R4
MUL    R1, R1, R3 → R1 ← R1 * R3
STORE  X, R1      → M[X] ← R
```

Instruction Formats

- Comparison of Instruction Formats:

Type	No. of Addresses	Example	Complexity
Three-address	3	R1 = R2 + R3	High
Two-address	2	R1 = R1 + R2	Medium
One-address	1	AC = AC + X	Low
Zero-address	0	Stack based	Very Low
RISC	Fixed format	Register-based	Very Efficient

Comparison of RISC and CISC

ADITYA UNIVERSITY

S.No	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
1	Few instructions	Large number of instructions
2	Few addressing modes	Many addressing modes
3	Memory access only through Load/Store	Memory accessed directly in instructions
4	Operations performed within CPU registers	Operations can involve memory + registers
5	Fixed-length instruction format	Variable-length instruction format
6	Single-cycle execution (mostly)	Multi-cycle execution
7	Uses hardwired control unit	Uses microprogrammed control unit
8	Large number of registers	Fewer registers
9	Efficient pipelining	Complex pipelining
10	Simple instruction decoding	Complex instruction decoding
11	Focus on speed and performance	Focus on code compactness

Addressing Modes

ADITYA UNIVERSITY

- Addressing mode defines **how the operand of an instruction is obtained during execution**.
- It determines whether the operand is in **register, memory, or within the instruction itself**.

Instruction Format:

Mode	Opcode	Address

- **Mode** → Specifies addressing mode
- **Opcode** → Operation
- **Address** → Operand / address of operand

Types of Addressing Modes:

1. Implied
2. Immediate
3. Register
4. Register Indirect
5. Direct
6. Indirect
7. Relative
8. Indexed
9. Base Register
10. Auto Increment / Auto Decrement

Addressing Modes

ADITYA UNIVERSITY

1. Implied Addressing Mode

- In this addressing mode, the Operand is **implicitly specified** in the instruction.
- No address field is required.

Examples:

- CMA : Complement Accumulator ; Meaning: AC ← AC'
- CLC : Clear Carry; Meaning: C ← 0
- PUSH and POP

2. Immediate Addressing Mode

- In this addressing mode, the Operand is given directly in the instruction.

Example:

MOV A, #20; Meaning: A ←20

Operand = 20 (no need of memory access)

Addressing Modes

ADITYA UNIVERSITY

3. Register Addressing Mode

In this addressing mode, the Operand is stored in a **CPU register**.

Example:

MOV R1, R2; Meaning: R1 ← R2

4. Register Indirect Addressing Mode

- In this addressing mode, the Register contains address of operand.

Example:

MOV A, (R0);

- The CPU register hold the address of the operand; Therefore, the operand = M[R0].
- Operand = M[R0]
- Meaning: A ← M[R0]

Addressing Modes

5. Direct Addressing Mode

- In this addressing mode, the Address field directly gives memory location of operand.

Example:

MOV A, 2000;

- The operand is available in the memory location 2000, that is M[2000]
- Meaning: A ←M[2000]

6. Indirect Addressing Mode

- In this addressing mode, the Address field points to a location that contains effective address.

Example:

MOV A, (2000);

- Effective Address (EA) = M[2000]
- Operand is available in the effective address, that is M[EA]
- Meaning A ← M[EA]

Addressing Modes

ADITYA UNIVERSITY

7. Relative Addressing Mode

- In this addressing mode, Uses Program Counter (PC).

Formula: EA = PC + Address field

Example: MOV A, X ;

- Meaning: A ← M[PC + X]; Here X is the Address field.
- PC = 826, Address = 24 then EA = 850.

8. Indexed Addressing Mode

- In this addressing mode, Uses Index Register (XR).

Formula: EA = XR + Address field

Example: MOV A, X(XR)

- Meaning: A ← M[XR + X]; Here XR is Index Register and X is the Address field.
- XR = 100, Address = 500 then EA = 600.

Addressing Modes

ADITYA UNIVERSITY

9. Base Register Addressing Mode

- In this addressing mode, Uses Base Register.

Formula: EA = Base Register + Address field

Example: MOV A, X(BR) : A ← M[BR + X]

10. Auto Increment Addressing Mode

- The Register contains address
- After access this mode of instruction, the register increments the address.

Example: MOV R1, (R0)+ : R1 ← M[R0], R0 ← R0 + 1

11. Auto Decrement Addressing Mode

- Register is decremented first, then used.

Example: OV R1, -(R0) : R0 ← R0 - 1, R1 ← M[R0]

Data transfer and Data Manipulation Instructions

ADITYA UNIVERSITY

- **Classification of Instructions:** Instructions are divided into three categories:
  - Data Transfer Instructions
  - Data Manipulation Instructions
  - Program Control Instructions
- **Data Transfer Instructions:**
  - Used to move data from one location to another and data is not modified
  - In general, the data transfer may be taken as follows.

    Memory ↔ Registers

    Register ↔ Register

    Processor ↔ I/O devices
  - The common data transfer instructions are mention in the below table:

Typical Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

Data transfer and Data Manipulation Instructions

ADITYA UNIVERSITY

Data Manipulation Instructions:

- Data manipulation instructions perform operations on data and provide the computational capabilities for the computer.
- The data manipulation instructions are usually divided into three basic types:
  1. Arithmetic instructions
  2. Logical and bit manipulation instructions
  3. Shift instructions

Arithmetic instructions:

- Used to perform athematic operations.
- There are four Arithmetic Operation can be performed by a basic computer: **Addition, Subtraction, Multiplication and Division.**
- The typical Arithmetic Instructions are mention in the table:

Typical Arithmetic Instructions

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

Logical and bit manipulation instructions:

- Used to perform logical operations using Boolean operations.
- Basic Logical Operations are: AND, OR, XOR
- Special Logical Operations are: set, clear, complement and disable.
- The typical Arithmetic Instructions are mention in the table.

Typical Logical and Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Shift instructions:

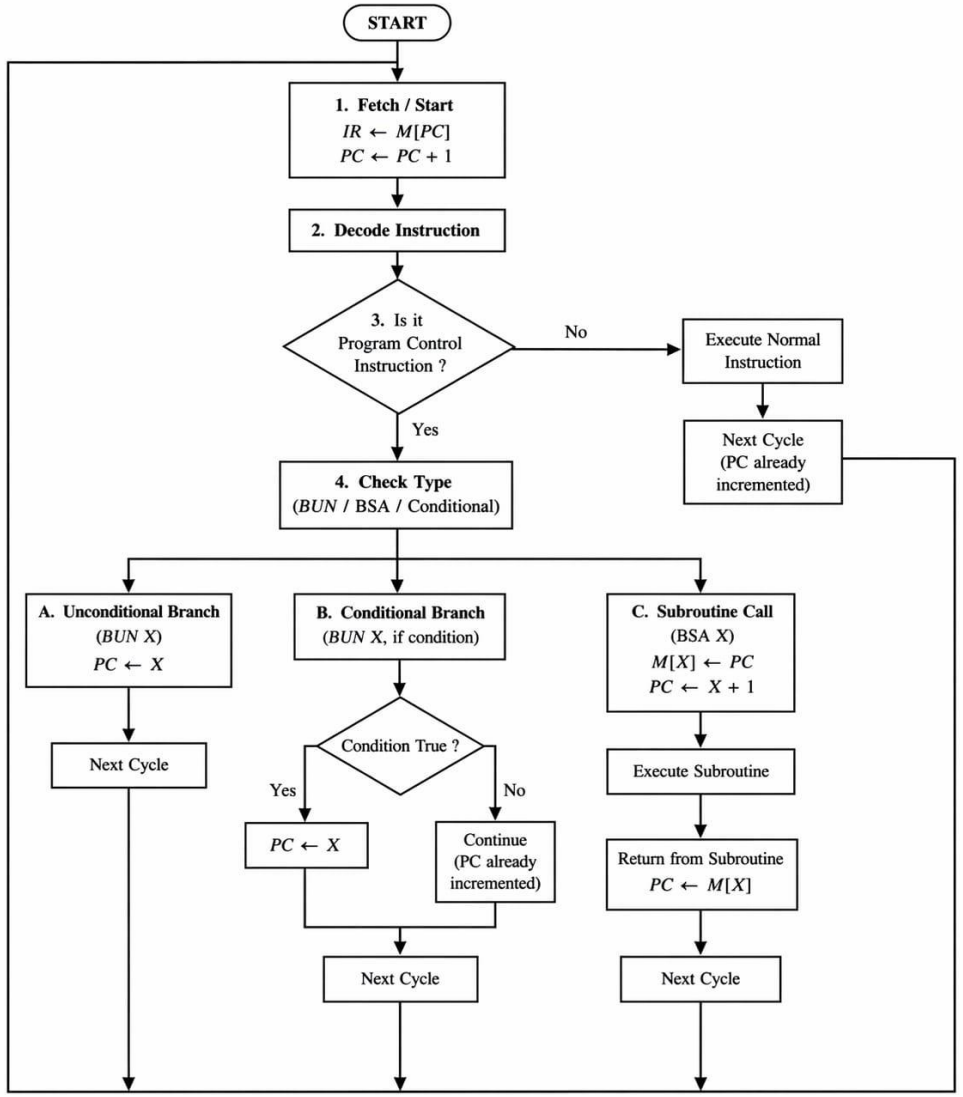
- Used to perform shift operations using Boolean operations.
- Types shift operations: Arithmetic shift, Logical Shift, Circular shift
- Direction of shift: Left shift and Right shit.
- The typical Arithmetic Instructions are mention in the table.

Typical Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Program Control

- **Program control instructions** are used change the sequence of execution of instructions instead of executing them sequentially.
- First, the instruction is fetched from memory using **IR ← M[PC]** and the Program Counter is incremented as **PC ← PC + 1**. and identify the instruction..
- Next, the control unit checks whether the instruction is a **program control instruction**.
  If it is not, the instruction is executed normally and the system proceeds to the next cycle.
- If it is a program control instruction, the type of instruction is identified as **Unconditional Branch (BUN), Conditional Branch, or Subroutine Call (BSA)**.
- In the case of an **unconditional branch**, the Program Counter is directly loaded with the new address (**PC ← X**), and execution continues from that location.
- In a **conditional branch**, the condition (based on AC or status flags) is checked. If the condition is true, the Program Counter is updated (**PC ← X**). Otherwise, execution continues with the next sequential instruction since the PC is already incremented.
- In a **subroutine call**, the return address is first saved in memory (**M[X] ← PC**), and then control is transferred to the subroutine (**PC ← X + 1**). After executing the subroutine, the original address is restored using **PC ← M[X]**, and execution returns to the main program.



Typical Program Control Instructions

ADITYA UNIVERSITY

Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

Conditional status Flags

- Conditional branch instructions use **status flags (bits)** which are generated by ALU
- There are 4 status flags and all stored in the **status register**.

  1. Z (Zero flag) → Set when result = 0 → Used for *Branch if Zero*

  2. S (Sign flag) → Set when result is negative (MSB = 1) → Used for *Branch if Negative*

  3. C (Carry flag) → Set when carry/borrow occurs → Used for *Branch if Carry*

  4. V (Overflow flag) → Set when overflow occurs → Used for *Branch if Overflow*

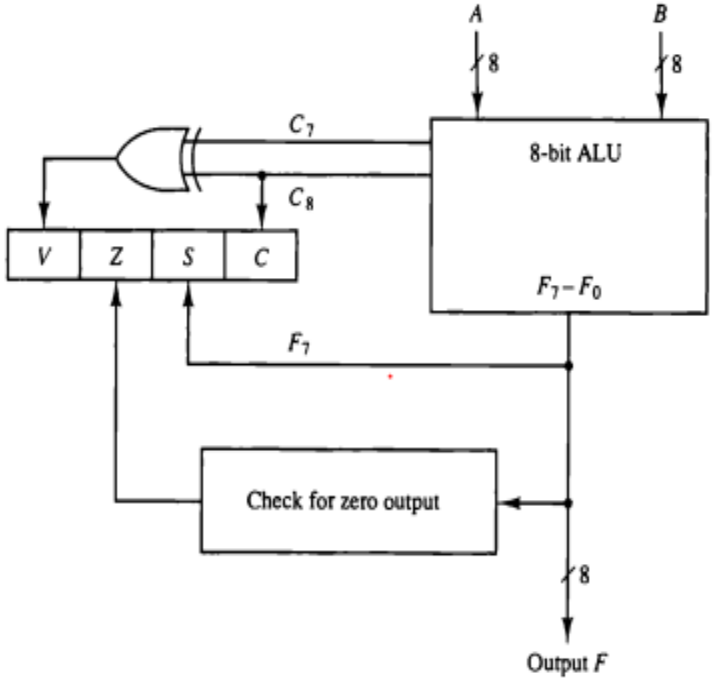


Figure 8-8 Status register bits.

- Control unit checks the required flag.
- If **condition is true (flag = 1) → PC ← X (branch taken)**
- If **condition is false** → Execution continues normally

Micro-programmed Control

- The control unit can be implemented in two ways

  1. Hardwired control unit: Logic is implemented with Gates, Decoders, Encoders, MUX, and De-MUX

  2. Microprogrammed control unit: Logic is implemented with a **microprogram**, i.e., stored in the control memory

- **Microprogrammed Control:**

- Control information is stored in control memory.
- Control memory is programmed to initiate the required sequence of micro-operations.

- **The key characteristics are**

1. The speed of operation is low when compared with hardwired
2. Less complex
3. Less expensive
4. .Flexibility to add new instructions

- **Examples** of CPUs with microprogrammed control units are the Intel 8080, Motorola 68000, and any CISC CPUs.

Microprogrammed Control: Definitions

- **Microprogram Control Unit:**

  A control unit in which control signals are stored in memory instead of being generated directly.
- **Control Memory:**

  A special memory (usually ROM) that stores all the microinstructions required for operation.
- **Control Word:**

  A group of binary bits (1's and 0's) that represents control signals at a particular time and each control word corresponds to **one microinstruction**.
- **Microinstruction:**

  It specifies the control signals needed to perform a small operation inside the CPU.
- A sequence of microinstructions forms a **microprogram**.
- The microprogram controls the **execution of instructions and overall system operations**.

Microprogrammed control organization

- **Control Memory (ROM)** stores all microinstructions required for system operation. When an address is received from the Control Address Register, it fetches the corresponding microinstruction and sends it to the Control Data Register for execution. The organization of Microprogram control is shown in Fig. below.

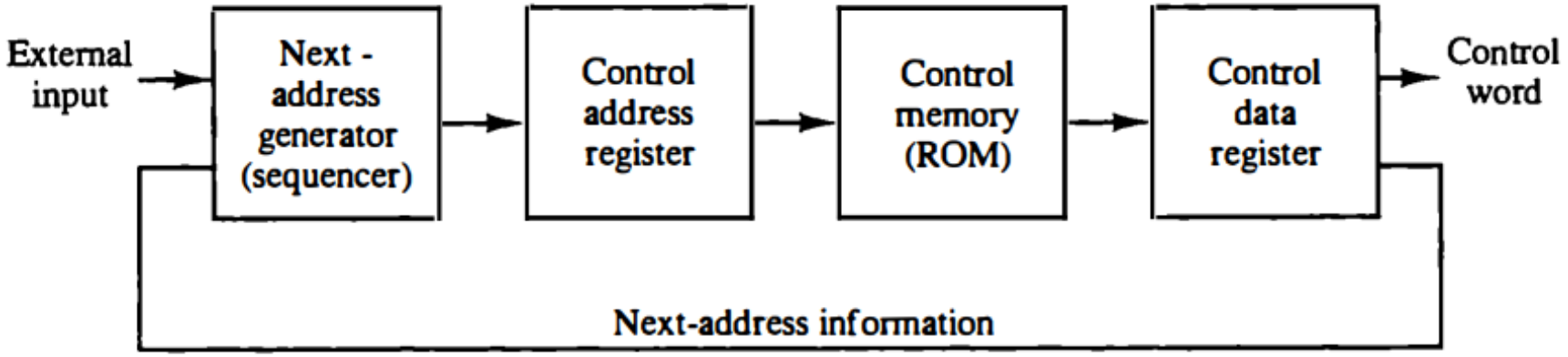


Fig. Microprogrammed control organization

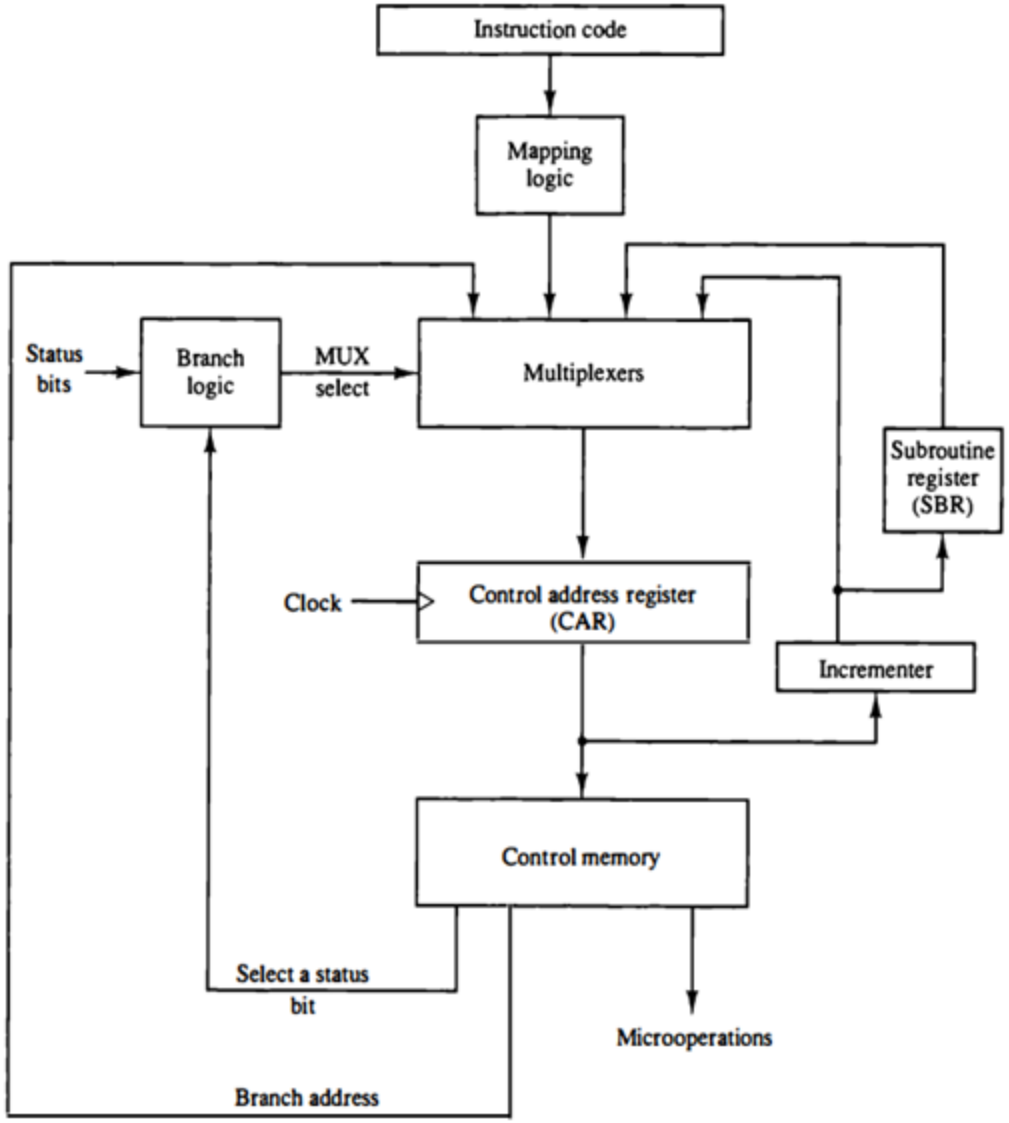
- Sequencer generates **next address**
- Address is stored in **Control Address Register (CAR)**
- CAR sends address to **Control Memory (ROM)**
- Microinstruction is loaded into **Control Data Register (CDR)**
- CDR outputs **control word** to CPU
- Sequencer uses feedback to decide **next instruction address**
- **Control Memory accesses** the specified location and outputs the corresponding microinstruction

Control Memory

- **Control Memory (ROM)** stores all microinstructions required for system operation. When an address is received from the Control Address Register, it fetches the corresponding microinstruction and sends it to the Control Data Register for execution.
- A special memory used in the microprogram control unit is called Control Memory.
- Stores all microinstructions (control words).
- Each memory location contains one microinstruction.
- Implemented using ROM (Read-Only Memory).
- Receives address from the Control Address Register (CAR).
- Outputs the corresponding microinstruction.
- Provides control signals to CPU operations.
- Controls execution of instructions in a step-by-step manner.

Address Sequencing

- Address sequencing is the process of **selecting the address of the next microinstruction** to be executed in the control memory.
- The next address of microinstruction is determined by using a multiplexer (MUX).
- The MUX selects one of the following inputs:
  - Next sequential address (from incrementor)
  - Branch address (from control memory)
  - Mapping logic output (starting address of instruction)
  - Subroutine return address (from SBR)
- The selected address is loaded into the Control Address Register (CAR).
- CAR sends this address to control memory to fetch the next microinstruction.



Address Sequencing continue.,

The **MUX selects the next microinstruction address** from different sources in the control unit

- **Next Sequential Address (from Incrementor):**
  - Obtained by incrementing current address (CAR + 1)
  - Used for normal step-by-step execution
- **Branch Address (from Control Memory):**
  - Provided by the current microinstruction
  - Selected when a branch condition is true (based on status bits)
  - Used to jump to another location
- **Mapping Logic Output (Starting Address):**
  - Generated from the opcode of the instruction
  - Gives starting address of microprogram
  - Used at the beginning of instruction execution
- **Subroutine Return Address (from SBR):**

•Stored in Subroutine Register

•Used to return after subroutine execution

Mapping of Instruction

- Mapping is the process of **converting the opcode of an instruction into the starting address of its microprogram** in control memory.
- A computer instruction consists of: Opcode (operation code) → e.g., 1011 and Address field
- The opcode is used to generate the starting microinstruction address.
- Opcode is mapped into control memory address.
- This address points to the first microinstruction of that instruction

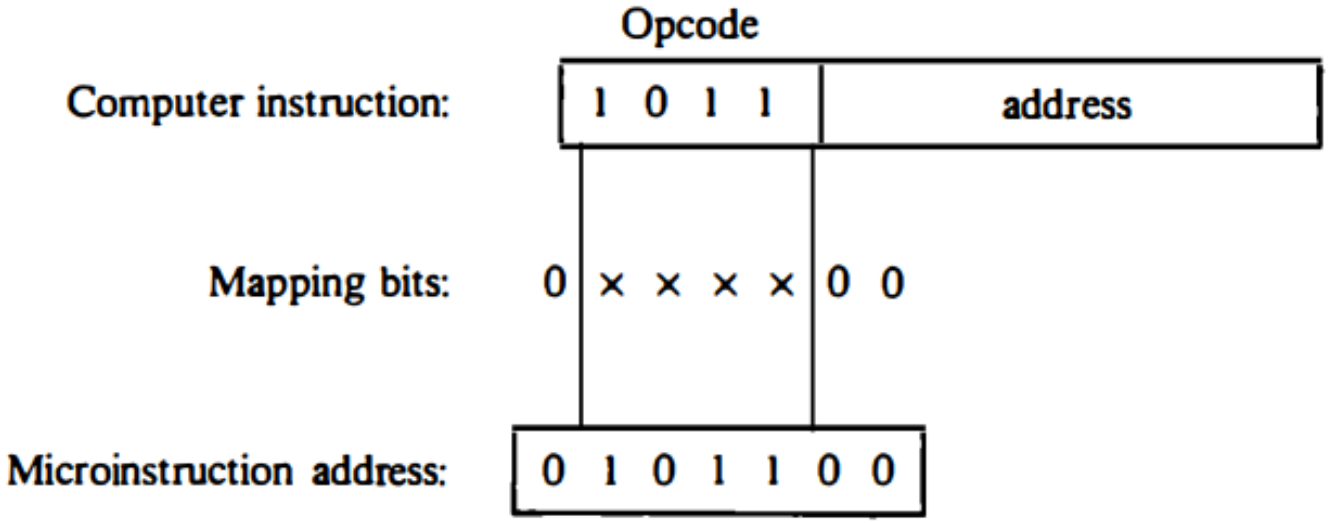


Fig. Mapping of microinstruction

Design of Control Unit and Decoding of Microinstruction Fields

- Control unit generates **control signals** using microinstructions
- These **microinstructions** are stored in control memory (ROM)
- Each instruction is executed as a sequence of micro-operations
- To reduce number of bits, the **Control signals** are grouped into fields and each field is decoded using decoders.
- The microinstruction is divided into three fields: F1, F2, F3
- Each field is given to a 3×8 decoder and each decoder generates control signals.

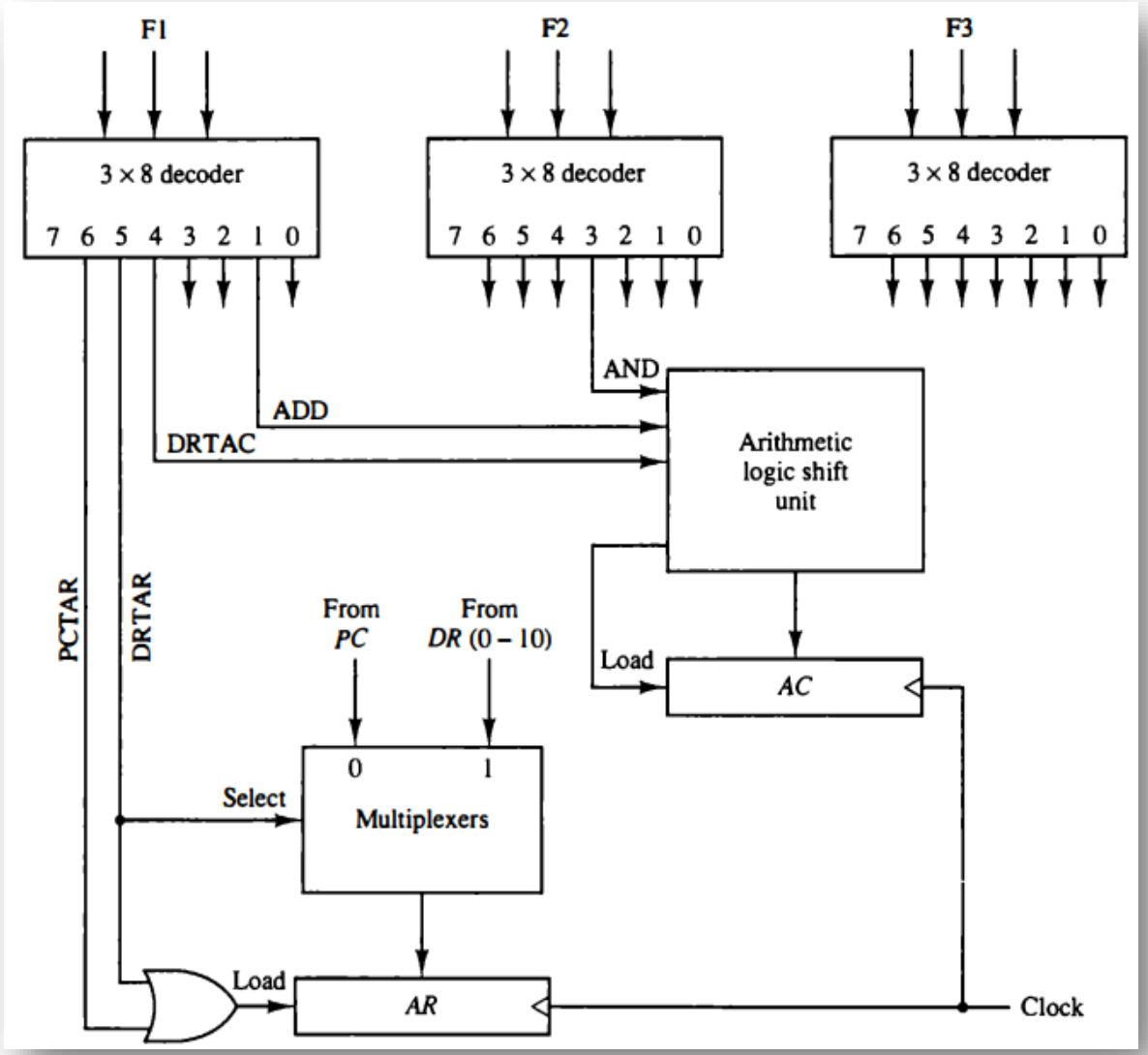


Fig. Decoding of Microinstruction Fields

Decoding of Microinstruction Fields continue

- .The outputs of **Decoder** represent micro-operations as:

  ADD → Perform addition

  DRTAC → Transfer data from DR to AC

  PCTAR → Transfer PC to AR

  DRTAR → Transfer DR to AR

- **Arithmetic Logic Shift Unit (ALU):**

It receives inputs from decoder outputs and performs operations like: ADD, AND and result is sent to AC (Accumulator).

- **Accumulator (AC):** It stores the result of ALU operation.
- **Multiplexer (MUX):** It selects input for Address Register (AR) from the inputs of : PC and DR.
- **Address Register (AR)**, receives selected data from MUX

Also receives data directly from: PC (PCTAR) and DR(DRTAR)

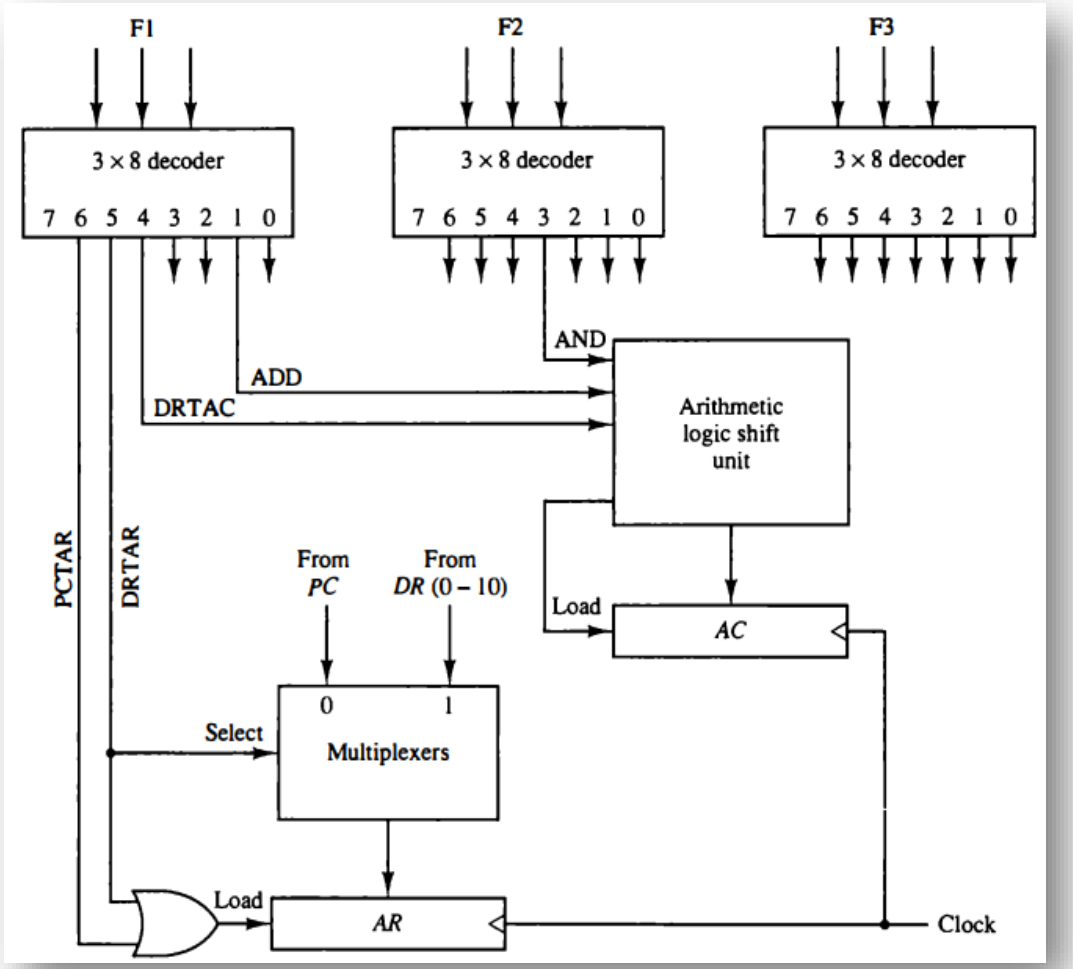


Fig. Decoding of Microinstruction Fields

Fields of Microinstruction Format

- **The format of microinstruction is shown in the figure.** It consists of several parts:
  - Three fields: **F1, F2, F3**
  - **CD (Condition field)**
  - **BR (Branch field)**
  - **AD (Address field)**
- **Three micro-operation fields: F1, F2, F3 (Each = 3 bits) :** Used to select different operations in the system. Each field is decoded to generate control signals for data transfer and ALU operations.

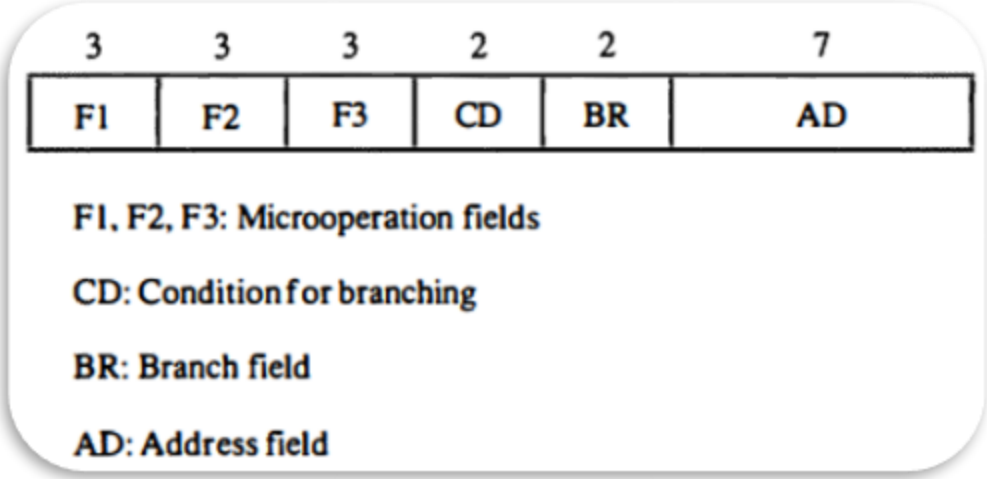


Fig. Microinstruction Fields Format

- **CD (Condition Field) Size = 2 bits:** This field specifies the condition for branching. It checks status bits like zero or carry to decide whether branching should occur.
- **BR (Branch Field) Size = 2 bits** :Specifies **type of branch:** No branch, Conditional, Unconditional or Subroutine
- **AD (Address Field) Size = 7 bits:** This field contains the address of the next microinstruction. It is used when a branch or jump operation is required.

Microprogram Sequencer

- The microprogram sequencer is responsible for generating the next address of microinstruction
- The internal organization of a microprogram sequencer used in a microprogrammed control unit is shown in Figure.
- The key components of this sequencer are:
  - **MUX (Multiplexer)** → selects next address
  - **CAR (Control Address Register)** → holds current address
  - **Control Memory** → stores microinstructions
  - **Incrementor** → generates next sequential address
  - **SBR (Subroutine Register)** → stores return address
  - **Input Logic** → controls selection signals
  - **MUX2 (Test logic)** → checks conditions
- This unit controls **how microinstructions are executed step-by-step** by selecting the correct next address.

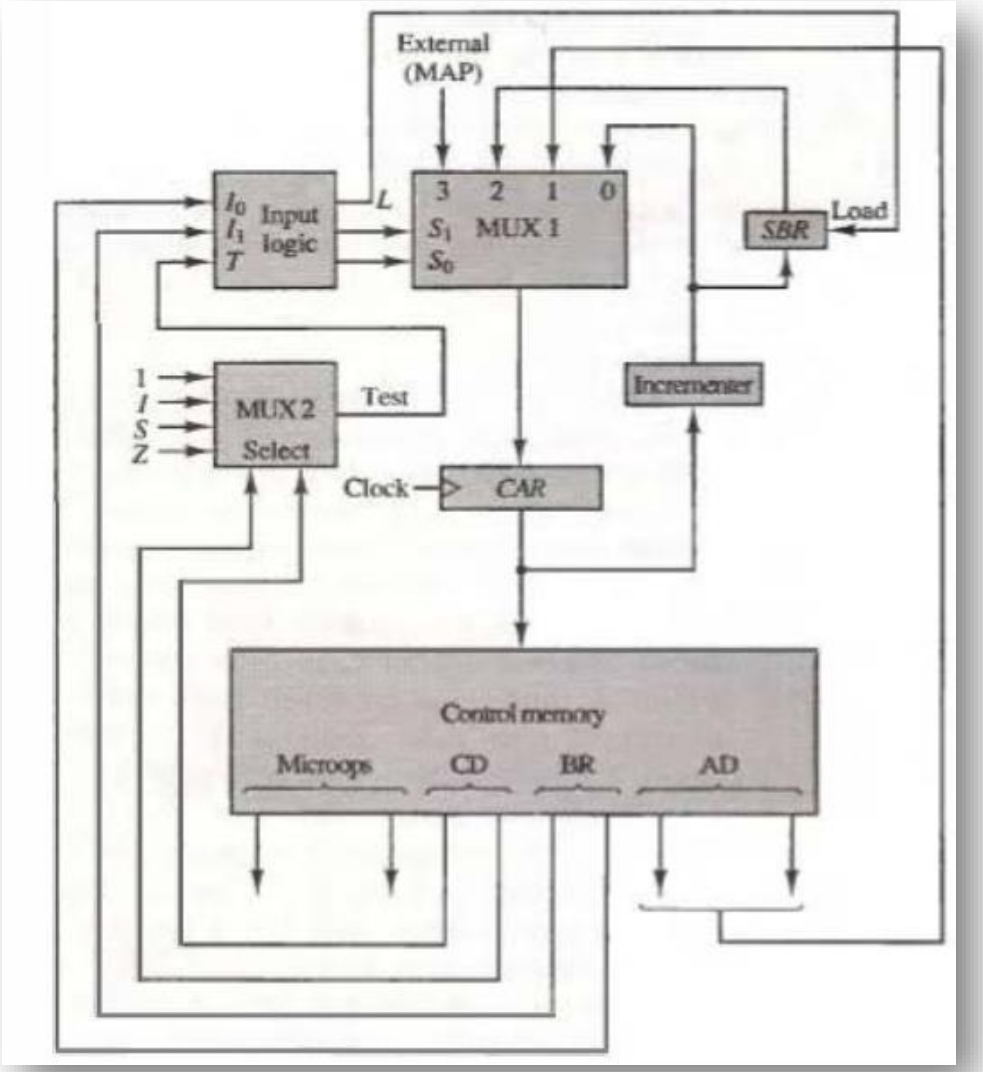


Fig. Internal organization of Microprogram Sequencer

Microprogram Example

- A microprogram example explains how a computer instruction is executed using a sequence of microinstructions stored in control memory.
- The control unit generates control signals step-by-step using these microinstructions.
- There are five subtopics

1. Computer Configuration
2. Instruction Formats
3. Symbolic microinstruction
4. Symbolic microprogram
5. Binary microprogram

Microprogram Example

1. Computer Configuration:

This diagram shows how a computer executes instructions using microprogrammed control.

1. Memory (2048 × 16)

2. Registers: PC, AR, DR and AC.

3. Multiplexers (MUX): Select input from different sources

4. ALU (Arithmetic Logic Shift Unit)

Performs arithmetic operations like: ADD, AND and SHIFT

5. Control Unit: It Contains

Control Memory (128 × 20) → stores microinstructions

CAR (Control Address Register) → holds microinstruction address

SBR (Subroutine Register) → stores return address

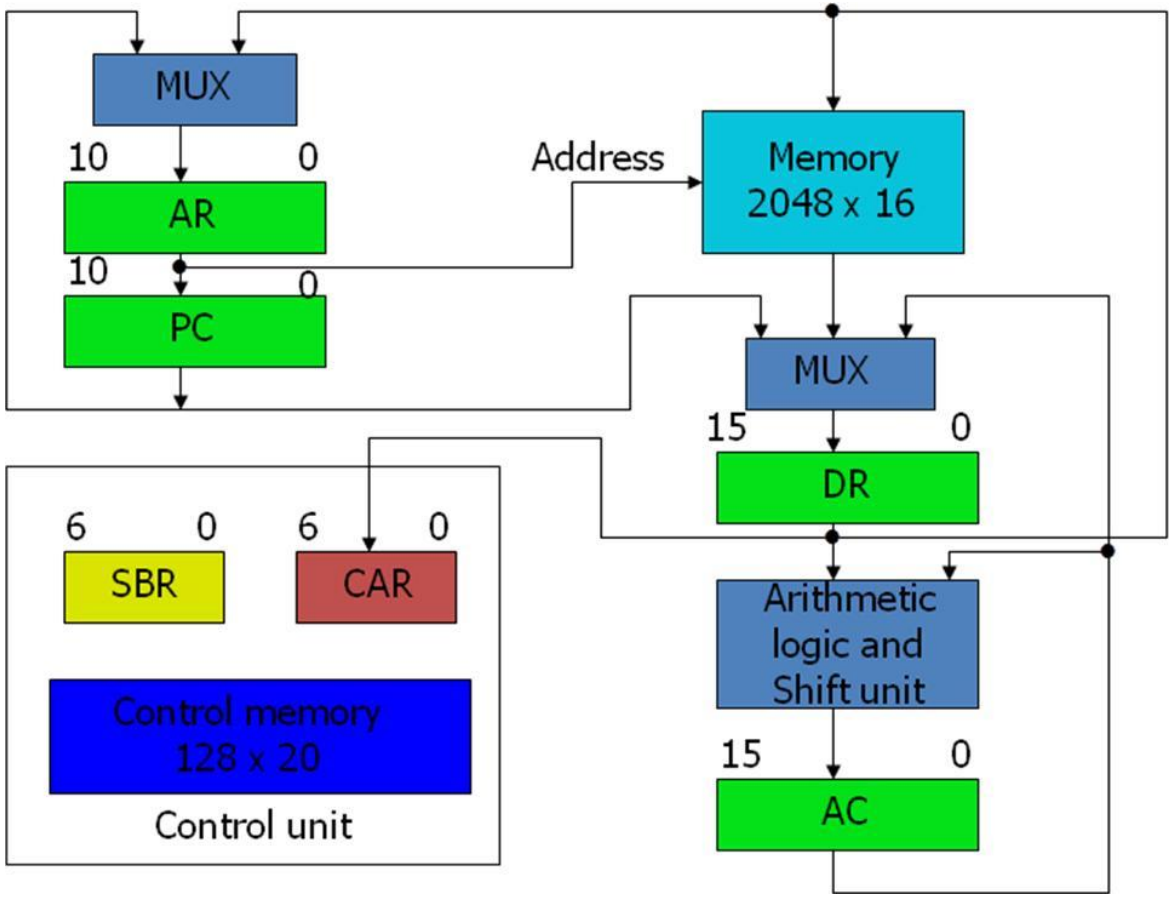


Fig. Computer hardware configuration

Microprogram Example

2. Instruction Format

The instruction format consists of **Opcode (4 bits)** and **Address field (12 bits)**.

Examples:

ADD → AC ← AC + M[EA]

BRANCH → PC ← EA

STORE → M[EA] ← AC

EXCHANGE → AC ↔ M[EA]

3. Microinstruction Format

A microinstruction is 20 bits long and consists of the following fields:

F1, F2, F3: Microoperation fields

CD: Condition field

BR: Branch field

AD: Address field

These fields control operations, branching, and the next address selection.

4. Symbolic Microinstruction

Symbolic microinstructions use simple mnemonics instead of binary codes.

Examples:

DRTAC: DR → AC

PCTAR: PC → AR

ADD: AC ← AC + DR

5. Symbolic Microprogram

A symbolic microprogram is a sequence of symbolic microinstructions used to execute an instruction.

Example (Fetch routine):

AR ← PC

DR ← M[AR]

PC ← PC + 1

5. Binary Microprogram

The symbolic microprogram is converted into binary form and stored in control memory. Each microinstruction is represented using binary values of **F1, F2, F3, CD, BR, and AD fields**.